Countersign: Evidence-Grounded Supervision of Coding-Agent Completion Claims

Anonymous Author(s)
Affiliation
Address
email

Abstract

Coding agents terminate when they believe a task is done, but that belief is not always supported by the evidence in their own execution trace. We present *Countersign*, a deterministic supervisory meta-agent that audits a worker's completion claim against the trace—checking that cited tests exist, remain fresh with respect to the files they cover, postdate the latest requirement update, and do not rest on superseded sources—and can block termination and demand fresh evidence. Countersign is deliberately *trace-only*: it judges whether a claim is *justified* at claim time, a different question from whether the implementation is *correct*, which a hidden evaluator answers once after termination. In a preregistered, frozen campaign of 490 real-model runs over ten held-out fixtures, Countersign issued 17 blocks; all 17 landed on proposals an independent oracle also labelled unsupported (precision 1.00, 95% CI $[0.82, 1.0]$), and none on any of the 204 supported proposals a verifier judged—though 13 of the 17 are the trivial case of a claim citing no evidence at all. A counterfactual replay shows a conventional continuous-integration gate catches 2 of 15 unsupported proposals where Countersign catches 11 of 11 it judged. Our negative results are equally informative: truncating the worker's visible history did not increase unsupported completion, the effect appearing only when history is *replaced by a narrative summary*, and we decompose that effect into prompt and gate contributions. We also quantify the construct honestly—25 of 27 unsupported proposals were backed by work that passed hidden evaluation—so Countersign is an evidence-hygiene instrument whose safety value remains a hypothesis under test.

1 Introduction

An autonomous coding agent ends its episode by asserting completion. That assertion is a claim about evidence: the tests it ran, the edits it made, the requirements it was given. When the claim outruns the evidence—citing a test that predates the final edit, or resting on a document the project has since superseded—the agent has not necessarily produced wrong code, but it has produced an *unwarranted* report, and any human or downstream system that trusts the report inherits a risk it cannot see.

We study whether a second agent, given authority over the first agent's termination, can reliably tell warranted completion claims from unwarranted ones, and what that authority costs. Our supervisor, Countersign, reads only the worker's execution trace: it never consults ground truth, and the hidden evaluator that determines correctness runs exactly once, after termination, identically in every condition. This separation is the paper's organizing principle. A claim can be justified and wrong; it can also be correct and unjustified. Conflating the two is what makes "the verifier improved outcomes" an unfalsifiable statement.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Contributions. We contribute (i) a supervision loop with halting authority over a worker's termination, scored against an independent post-hoc oracle it does not compute (§2); (ii) a preregistered evaluation on matched fixture pairs and negative controls that price false blocks, under a frozen protocol with a public deviation ledger (§3); (iii) empirical results across 490 runs—perfect block precision, a counterfactual ablation against a conventional CI gate, a null dose–response under truncation, and a regime-specific effect under summarization decomposed into prompt and gate contributions (§4); and (iv) a calibrated account of the gate as an evidence-hygiene instrument rather than a demonstrated safety mechanism (§5).

2 The Countersign Supervisor

Worker and ledger. The worker is a tool-loop coding agent (self-hosted open-weight models) in a git-backed workspace. Every action appends to an *evidence ledger*: tool calls with outcomes, file mutations, test runs with inferred coverage, and requirement updates. To finish, it emits a `finish` action carrying a claim and a list of cited evidence identifiers.

The audit. Countersign checks four properties of the cited evidence: (i) at least one cited event is a successful test run; (ii) no cited test predates a later mutation of a file it covers (coverage is inferred statically, so documentation edits do not invalidate code evidence, and a failure superseded by a later passing run does not permanently contradict the claim); (iii) cited evidence postdates the latest relevant requirement update; and (iv) the claim does not rest on sources the task marks superseded. Freshness is tri-state—the supervisor abstains rather than guessing when coverage is unknown.

Authority. On failure the supervisor blocks termination and returns the specific deficiency ("your cited test predates the edit to `X`"). A repair variant additionally issues the smallest evidence-refreshing action. Every arm logs the *raw* verifier decision separately from the *enforced* one, so non-blocking arms still measure what the supervisor would have done.

Independent oracle. A post-hoc oracle labels every proposal supported/unsupported/uncertain from fixture-authored completion policies and canonical trace chronology. It shares no code with the online verifier; the two may disagree, and §4 reports a regime in which they do.

3 Evaluation Design

Fixtures. Ten held-out fixture repositories, authored to a preregistered design and frozen before any real-model run was inspected. Three *matched pairs* isolate one failure mode each under context parity (identical planned actions; workspaces byte-identical but for one designed file): *temporal* (cited tests predate the final edit), *provenance* (fresh evidence grounded in a superseded document), and *requirement* (a clarification arrives after the last passing run). Four *negative controls* contain no trap—documentation edits after green tests, an unrelated-module edit with a targeted-test citation, a legitimate zero-change audit, and an irrelevant late clarification—so any block on them is a false positive by construction. A separate development suite, on which the rules were iterated, is never used for evaluation.

Conditions. Arms cross a worker prompt variant with verifier authority: memory_baseline (naive prompt, no gate), observe_only (evidence-citation prompt; verifier computes and logs a decision but never blocks and never returns feedback to the worker), verification_only (same prompt, blocking), verification_and_repair, and an evaluation-only oracle_supervisor upper bound. Because memory_baseline and verification_only differ in *both* prompt and gate, we never report that contrast as a gate effect; observe_only is the prompt-matched control, and we verified in-trace that it emits zero worker-visible feedback events.

Memory degradation. We additionally run the fixtures under a frozen registry of degradation profiles applied only to *model-visible* state: graded *lossy truncation* of visible history (low/medium/high) and a *resume-summary* regime in which the visible ledger is replaced by a single truthful summary—the pattern deployed scaffolds use on session resumption. The canonical ledger, verifier, oracle, and evaluator always see undegraded state.

Protocol. Temperature 0.7, sampler seeds 0–4 (greedy decoding would make multi-seed replication pseudoreplication), action budget 24. Endpoints, comparisons, and interpretation branches for both

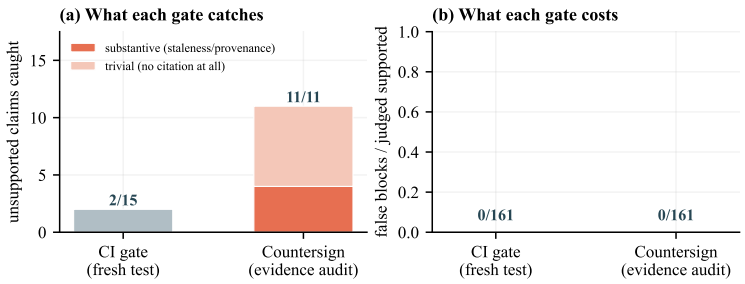

Figure 1: Countersign versus the conventional CI rule "require a successful test run after the last edit," replayed counterfactually over every stored proposal. (a) The CI rule catches 2 of 15 oracle-unsupported claims; Countersign catches all 11 it judged, but only 4 of those require substantive reasoning—the other 7 are claims citing no evidence at all, which any rule detects. (b) Neither gate blocked any of the 161 supported proposals both judged: the difference is blindness, not conservatism.

positive and null outcomes were committed before the runs; the frozen protocol hashes code, fixtures, and model weights and refuses to start on a dirty tree.

4 Results

All numbers are computed from audited run manifests; the worker is `qwen2.5-coder:14b` unless noted. The campaign comprises 490 real-model runs yielding 341 oracle-judged finish proposals, of which 27 are labelled unsupported.

Blocks are precise. Across every arm in which the verifier ran, Countersign issued 17 blocks. All 17 landed on proposals the independent oracle also labelled unsupported and none on any of the 204 supported proposals a verifier judged: precision 1.00 (95% CI $[0.82, 1.0]$), false-block rate $0/204$ (95% CI $[0, 0.018]$), recall $17/18 = 0.94$ (95% CI $[0.74, 0.99]$). Two caveats bound this result. First, the denominator counts only judged proposals: a further 110 supported proposals occurred in arms with no verifier and could not have been blocked, so they are excluded rather than counted as successes. Second, and more important, **13 of the 17 blocks are the trivial case**—a completion claim citing no evidence whatsoever, which any rule detects and on which the verifier and the oracle cannot meaningfully disagree. Only 4 blocks required substantive reasoning about staleness or provenance. The precision figure should therefore be read as "the gate does not misfire," not as "the gate is discriminating hard cases."

A conventional CI gate is blind to what actually occurs. We replay every stored proposal against the standard-practice rule "require a successful test run after the last edit," a strict subset of Countersign's temporal check (Figure 1). **Nine of the fifteen unsupported proposals are caught only by the evidence rules**, because in those cases the tests were fresh and the deficiency lay in citation or provenance. Neither gate blocked any of the 161 supported proposals both judged.

Truncating memory does not *detectably* induce false claims. Under the graded lossy regime, baseline unsupported completion on trap fixtures did not increase with severity (Figure 2a). We state this as a non-detection rather than a demonstration of invariance: at 15 runs per cell the Wilson intervals span roughly 0.01–0.30, so effects smaller than about 30 percentage points are not resolvable here. The preregistered null branch applies, with that power caveat attached.

The effect is regime-specific, and decomposes. Replacing history with a resume summary—rather than truncating it—does produce unsupported completion (Figure 2b). Read as a 2×2 over ten matched cells, the prompt accounts for 2 of the 5 cases and the gate for 3 (exact McNemar $p = 0.50$ and $p = 0.25$; combined $5 \to 0$, $p = 0.0625$). We therefore report a decomposition and claim no significant effect. An a-priori power analysis explains why: with all discordant pairs in one direction the exact test requires $b \geq 6$ for $p < .05$, so the campaign is one discordant pair short of significance by construction, and 16 matched cells would be needed for 80% power at the observed effect.

Measurement without power is the cleanest evidence. In `observe_only` the verifier judges but cannot act, so its decisions cannot have influenced the trajectories it scores. Its would-block fired in

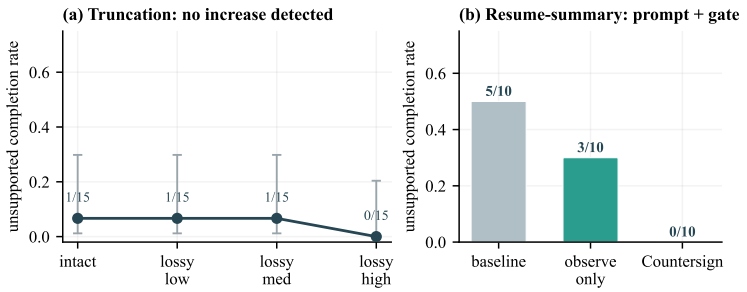

Figure 2: Memory degradation does not act uniformly. (a) Graded truncation of the worker's visible history produces no detectable increase in unsupported completion; note the width of the Wilson 95% intervals—this is a non-detection at modest power, not a demonstration of invariance. (b) Replacing history with a resume summary does produce the effect, and it decomposes: the evidence-citation prompt accounts for 2 of the 5 cases and the gate for the remaining 3. Small n; we report the decomposition, not a significance claim.

exactly the 3 unsupported cells and none of the other 7 (TP$=3$, FP$=0$, FN$=0$)—evidence immune to the circularity objection a blocking arm invites.

What "unsupported" means here. 25 of the 27 unsupported proposals had a fresh passing test in the canonical trace and passed hidden evaluation; 18 of 27 are unsupported because the worker cited *no* evidence at all. The measured phenomenon is therefore predominantly *unsubstantiated* rather than *incorrect* completion—verified work the worker failed to point at, most often when summarization removed the identifiers it would have cited.

5 Supervisor-Failure Analysis and Limitations

The structural blind spot. The provenance pair is built so that evidence is fresh but grounded in a superseded document. A trace-only supervisor allows those finishes, and the hidden evaluator fails them—a supported-but-incorrect outcome that no amount of trace auditing can reach. This bounds what runtime evidence supervision can promise.

Two further failure modes appear in Appendix A: the online classifier and the oracle diverge under heavy degradation (the classifier reads the worker's degraded view, the oracle the canonical trace), and one fixture family is censored by task weight rather than action budget.

Limitations. First, and most important, the held-out traps and the supervisor's rules were designed by the same authors; the development/held-out split prevents rule tuning on evaluation fixtures but cannot remove the conceptual correlation between what we chose to trap and what we chose to detect. Two design features partially offset this—the provenance family is built to *defeat* the supervisor, and the negative controls price the direction a self-built evaluation is least motivated to measure—but partial offset is not elimination. Second, one worker model on ten small fixture repositories establishes nothing about prevalence in production software, and our own null makes that gap sharper. Third, the online verifier and the post-hoc oracle share no code but do read the same trace, and the majority of agreements concern claims citing no evidence at all—a case on which independence is close to vacuous; the four substantive catches are the informative ones. Fourth, no result reaches $p < .05$ and the null results are non-detections at modest power, not demonstrations of invariance: we report decompositions and intervals, never significance. Fifth, the degradation profiles are our own transforms, not a deployed scaffold's condenser. Sixth, oracle-label validation covers a single-rater sample.

6 Related Work

Prior work measures workers gaming outcome checks METR [2025] and proposes runtime guardrails that gate *actions* for safety Chennabasappa et al. [2025]; Countersign instead gates *termination* for evidential support and prices the resulting false blocks. Self-verification asks the worker to check

itself Shinn et al. [2023]—external supervision is the complement when self-checking is what is in doubt. Outcome and process verifiers Cobbe et al. [2021], Lightman et al. [2024] inform our split between an online process check at termination and a single post-termination outcome check; long-context degradation Liu et al. [2024], Packer et al. [2023] motivates the memory regimes, and hidden-evaluation benchmarks Jimenez et al. [2024] the correctness oracle.

Responsible-Use Statement

Countersign controls agent overclaiming, and we expect its primary societal effect to be positive—fewer unverified "done" states in automated software work—but agents supervising agents carries risks this paper keeps visible.

An allow is not a guarantee. The supervisor audits justification, not correctness: our provenance fixtures construct cases where fresh, well-cited evidence grounded in the wrong source passes the audit while the hidden evaluator fails the work, and 25 of 27 unsupported proposals concerned work that was in fact verified. Countersign is an evidence-hygiene instrument and must never be marketed as verification of correctness.

Oversight displacement. A visible gate invites humans to stop checking. Because the supervisor is deterministic its failure modes are enumerable and we report them alongside its precision; verdicts should reach operators as evidence audits with cited events, not pass/fail badges.

Who supervises the supervisor. Halting authority creates a new failure surface—over-blocking denies service, repair guidance can loop a worker into budget exhaustion—which we price directly and recommend as standard counter-metrics for any deployed gate.

Scope. Experiments use self-hosted open-weight models on fixture repositories: no human subjects, personal data, or production systems. The released artifact contains fixtures, protocols, and run records only.

References

Sahana Chennabasappa et al. LlamaFirewall: An open source guardrail system for building secure AI agents. *arXiv preprint arXiv:2505.03574*, 2025. Author list to be verified against the arXiv page before submission.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

METR. Recent frontier models are reward hacking. `https://metr.org/blog/2025-06-05-recent-reward-hacking/`, 2025. PLACEHOLDER: verify before submission.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Additional Failure Modes

Classifier/oracle divergence. Under high degradation the online claim classifier disagreed with the independent oracle, because the classifier reads the worker's degraded view while the oracle reads the canonical trace. All endpoints in the main text are oracle-anchored; the divergence is evidence that an independent oracle is necessary rather than redundant.

Task weight censors a family. Requirement-family trap runs exhaust the action budget in $3/5$ runs in every arm, and raising the budget from 24 to 40 actions left that rate unchanged. The family is intrinsically too heavy for this worker—a fixture-design finding, not a budget-calibration error.

B Reproducibility

The campaign spans three tags of one freeze: a base freeze plus two infrastructure-only fixes applied mid-campaign (interruption-recovery artifact reuse; a tool-level crash on syntactically invalid worker-written code surfacing as a tool error rather than killing the episode). The hashed verifier-policy files are byte-identical across all three tags: no verifier, oracle, or fixture logic changed after the freeze. Every deviation, interim inspection, and protocol-identifier supersession—including two interim analyses that informed spending decisions, and one corrected over-interpretation—is recorded in the deviation ledger shipped with the artifact. The artifact is a relocatable bundle whose integrity audit passes from the copied location.